

Coroutine TS ready issues (25 and 27)

Doc. no.	P1356R0
Revises	none
Date:	2018-11-08
Project:	Programming Language C++
Reference:	ISO/IEC TS 22277, C++ Extensions for Coroutines
Audience:	WG21
Reply to:	Gor Nishanov < gorn@microsoft.com >

Introduction

All proposed resolutions wording is relative to [N4775](#).

Table of content

- Ready Issues: [25](#) [27](#)

Ready Issues

[25](#). Allow unhandled exception escape the user-defined body of the coroutine and give it well defined semantics

Section: 11.4.4 [dcl.fct.def.coroutine] **Status:** Ready **Submitter:** Eric Niebler **Opened:** 2018-03-10 **Last modified:** 2018-05-05

[EWG Approved Rapperswil-2018/06/09]

[CWG Approved San Diego-2018/11/08]

Issue:

Currently, an unhandled exception can never escape the user-authored body of the coroutine with triggering undefined behavior.

```
{
    P p;
    co_await p.initial_suspend(); // initial suspend point
    try {
        F // user authored body
    } catch(...) { p.unhandled_exception(); }
    final_suspend:
    co_await p.final_suspend(); // final suspend point
}
```

An exception from `F` is captured by the try-catch and a customization point `unhandled_exception` is called, where, typically, an `exception_ptr` is created and propagated to the consumer awaiting on async task, or, in case of a generator, will be delivered to the user when they dereference the iterator.

Though the current behavior is perfectly reasonable for asynchronous scenarios, it is sub-optimal for synchronous generators. Capturing an exception, storing it in an `exception_ptr` and then rethrowing the exception during, say, iterator's `operator*` is a needless work if the desired behavior is to let the exception propagate to the caller whenever it asks for the next value.

Background information: When a coroutine is first invoked, any exception thrown before entering the user-authored body (for example allocation failure, promise constructor failure, failure to copy parameters, etc) propagates into the caller as with any normal function call. However, when the coroutine suspends and subsequently resumed, if an exception is thrown by an evaluation of `p.unhandled_exception()` or an evaluation of `co_await p.final_suspend()` the behavior is undefined. Note that a coroutine can be only resumed or destroyed when suspended at a particular suspend point. An exception leaving the coroutine at arbitrary point of the execution leaves the coroutine in the undefined state.

The proposed resolution is to eliminate the undefined behavior in the following manner:

1. Allow an exception to escape `p.unhandled_exception()` and, in that case, consider the coroutine to be at the final suspend point. Reminder: when a coroutine is at the final suspend point, the coroutine can only be destroyed and a call to member function `done()` of the coroutine handle associated with that coroutine returns true.
2. Eliminate possibility of an exception being thrown from evaluation of an expression `co_await p.final_suspend()` by stating that `final_suspend` member function of the coroutine promise and `await_resume`, `await_ready`, and `await_suspend` members of the object returned from `final_suspend` shall have non-throwing exception specification.

This resolution allows generator implementations to define `unhandled_exception` as follows:

```
void unhandled_exception() { throw; }
```

With this implementation, if a user of the generator pulls the next value, and during computation of the next value an exception will occur in the user authored body it will be propagate back to the user and the coroutine will be put into a final suspend state and ready to be destroyed when generator destructors is run.

Proposed Wording:

In subclause 11.4.4, add two new paragraph after paragraph 11.

12. If the evaluation of the expression `p.unhandled_exception()` exits via an exception, the coroutine is considered suspended at the final suspend point.
13. The expression `co_await p.final_suspend()` shall not be potentially-throwing ([except.spec]).

27. Make suspension in dynamic initializers of static and `thread_local` local variables ill-formed

Section: 8.3.8/2 [expr.await] **Status:** Ready **Submitter:** Richard Smith **Opened:** 2018-03-25 **Last modified:** 2018-03-25

[EWG Approved Rapperswil-2018/06/09]

[CWG Approved San Diego-2018/11/08]

Proposed wording:

Add underlined text to 8.3.8/2:

An *await-expression* shall appear only in a potentially-evaluated expression within the *compound-statement* of a *function-body* outside of a *handler* (Clause 18). In a declaration-statement or in the simple-declaration (if any) of a for-init-statement, an ~~await-expression~~ *await-expression* shall appear only in an initializer of that declaration-statement or simple-declaration. An *await-expression* shall not appear in the initializer of a block-scope variable with static or thread storage duration.